

23) Algorithm $\rightarrow$ space complexity $\rightarrow$ Auxiliary space + Input space
 $\rightarrow$ Total memory Requirement.

Explanation:-

Space Complexity refers to the total amount of memory required by an algorithm during its execution. It includes both an algorithm during its to store the input and the extra memory used by the algorithm.

Total memory Requirement = Input space + Auxiliary space.
 Space Complexity refers to the total amount of memory required by an algorithm during its execution.

Total memory Requirement = Input space + Auxiliary.

Input

1. Input space

Input space is the memory required to store the input given to the algorithm.

For example, if an array contains N elements.

Input space = $O(N)$

2. Auxiliary space

Auxiliary space is the extra memory used by an algorithm apart from the input data.

- Temporary variables.
- Extra arrays
- Data structure.
- Recursion stack.
- Dynamic memory.

3. Total memory Requirement.

The total memory required is calculated as

$$S(n) = I(n) + A(n).$$

where:

• $I(n)$ = Input Space.

• $A(n)$ = Auxiliary space.

Example:-

Consider an algorithm that sorts an array using an additional temporary array.

Input space = $O(n)$

Auxiliary space = $O(n)$

Therefore

$$S(n) = O(n) + O(n)$$

$$S(n) = O(n)$$

→ Q3)

Concept map:-

Algorithm → Space complexity → Auxiliary space → Recursion stack → memory usage.

Explanation

Space complexity measures the amount of memory used by an algorithm during execution.

In recursive algorithms, every function call is stored in the recursion stack. Therefore, the recursion depth directly affects the auxiliary space requirement.

Recursion Stack

When a recursive function calls itself, a new stack frame is created for every function call.

Each stack frame stores:

- Function parameters.
- Local variables.
- Return address.
- Current execution information.

When the base condition is reached, the function calls return one by one and the stack frames are removed.

Example : Factorial.

$$\text{factorial}(n) = n \times \text{factorial}(n-1).$$

For $n=5$.

factorial(5)

calls:

factorial(4)

then:

factorial(3)

then:

factorial(2)

then,

factorial(1)

The recursion depth is n.

Therefore:

$$\text{Auxiliary space} = O(n)$$

Example: Binary search.

In recursive binary search, the input size is divided by 2 at every step.

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots$$

The recursion depth is

$$O(\log n).$$

Therefore:

$$\text{Auxiliary space} = O(\log n).$$